

CQL Data Types

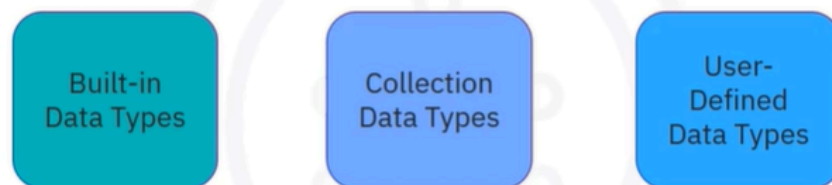
Sure! Let's talk about Cassandra's Collection Data Types in simple terms.

In Cassandra, collection data types are like special containers that help us group and store related pieces of information together in a single column. Imagine you have a box where you keep your favorite toys. Instead of having each toy scattered around your room, you can put them all in one box. Similarly, collection data types allow you to keep related data, like a list of a person's favorite books or a set of their hobbies, all in one place.

There are three main types of collections in Cassandra: **lists**, **maps**, and **sets**. A list is like a playlist of songs where the order matters, so you can have your favorite song first. A map is like a dictionary where each word (key) has a definition (value), such as a person's name paired with their phone number. Lastly, a set is like a collection of unique stickers; you can have one of each design, and they are automatically sorted.

CQL data types

Data types applied when declaring Cassandra tables:



- In CQL there are many data types, but they can be grouped into three main categories, built-in data types, collection data types, and user-defined data types. The user can choose any of them according to the requirements of the application and data model.

Built-in data types

Data Type	Data Type	
Ascii	Int	Blob: Are arbitrary bytes. A blob type is suitable for storing a small image or short string (1MB)
Boolean	Text	
Blob	Timestamp	Bigint: Are used for 64-bit signed long integers. This data type stores a higher range of integers compared to the “int” data type
Bigint	Timeuuid	
Decimal	Tinyint	Varchar: Is used for strings, and represents a UTF8 encoded strings
Double	Uuid	
Float	Varchar	

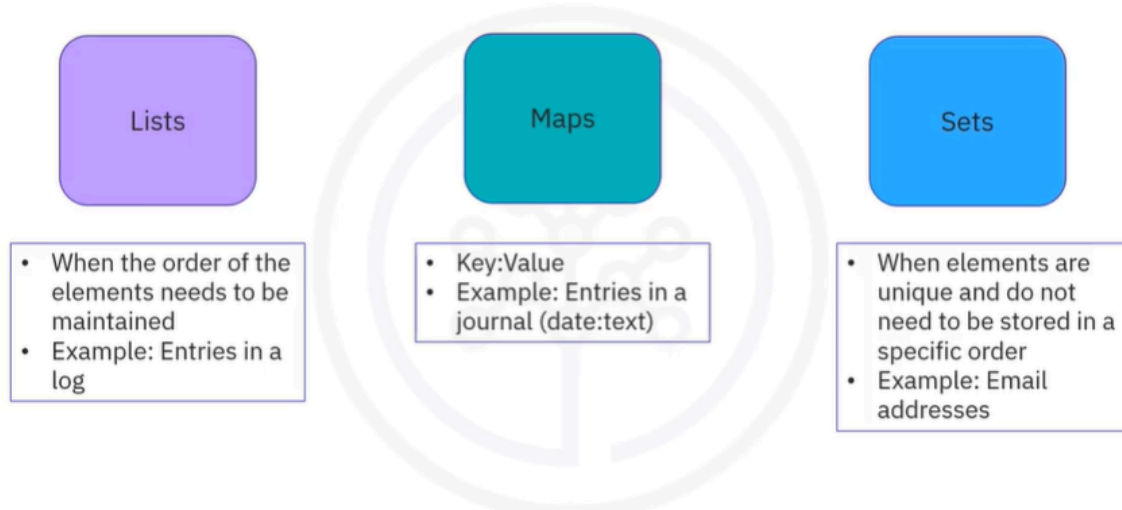
- Now let's discuss each of them in turn to understand what they mean. The built-in data types are basically pre-defined in Cassandra. The user can refer the variables to any of them. Besides regular data types like Ascii, Boolean, decimal, double, float, int, and text, which are fairly straightforward. There are some other types that maybe require a bit of explanation. Let's take the blob data type.
- Although Cassandra mainly stores text-based information, there is also the possibility to store blobs, which stands for binary large objects. Blobs are typically used to store images, audio, or other multimedia objects. While blobs represent a collection of binary data stored as a single entity, in Cassandra it is recommended that their size does not exceed 1 megabyte.
- Thus, you could store a small image or string using a blob. The bigint data type can be used for a 64-bit signed long integer. This data type stores a higher range of integers when compared to the int data type. The well-known

varchar is also available in Cassandra as a data type. It represents UTF8 encoded strings.

Collection data types

- Collections are a way to group and store data together
 - Example: A user has multiple email addresses
 - In a relational database: A many-to-one joined relationship between a 'users' table and an 'email' table
 - In Cassandra: No joins; Cassandra stores all the data in a collection column in the 'users' table
 - Data for collection storage should be limited: No unbounded growth
 - Not suitable for storing sent messages or sensor events stored every second
-
- Moving on to more complex data types, Cassandra provides collection types as a way to group and store data together in a column. For example, in a relational database, a grouping such as a user's multiple email addresses is related with a many-to-one joined relationship between a user's table and an email table.
 - Cassandra avoids joins between two tables by storing the user's email addresses in a collection column in the users table. Each collection specifies the data type of the data held. A collection is appropriate if the data for collection storage is limited.
 - If the data has unbounded growth potential, like messages sent or sensor events registered every second, do not use the collection data type. Instead, use a table with a compound primary key where data is stored in the clustering columns.

Collection data types



- Within the collection data types category, there are three data types. Lists, this Cassandra data type represents a collection of one or more elements in a table. List is to be used in cases where the order of the elements is to be maintained and a value is to be stored multiple times, such as entries in a log.
- Maps, this Cassandra data type represents a collection of key value pairs. Map is a data type that is used to store a key value pair of elements, such as entities in a journal entered using a date, and then text. And sets, this Cassandra data type represents a collection of one or more sorted elements in a table. Set is a data type that is used to store a group of elements. The elements of a set will be returned in a sorted order. An example would be a list of email addresses.

Collection data types: List

```
USE intro_cassandra;
```

Users table
PRIMARY KEY(userid)

```
ALTER TABLE users ADD jobs list<text>;
```

```
UPDATE users SET jobs = ['Walmart'] + jobs where username = 'Alaind@gmail.com'; //add the last job change to the list
```

```
UPDATE users SET jobs = jobs + ['Netflix'] where username = 'Alaind@gmail.com'; //add the last job change to the list
```

```
UPDATE users SET jobs[0]='Reiss' where username = 'Alaind@gmail.com'; //replaces Walmart with Reiss (lists start from 0)
```

- Let's go back to the users table and add a new column to the table called jobs, which is basically just a list of jobs. We would like to store the jobs in the order of their occurrences. Remember that the users table is a static table with its primary key consisting of the user id column.
- In Cassandra, we will store all the users jobs in a single column. Since we cannot perform joins, in this case, we will use the list type of the collection data types because we want to preserve the order of the jobs. Another reason is that a person can work at a specific company more than once, so uniqueness is not required.
- We can add a job in the list either at the beginning or end of the list or in a specific position.
- The entries in the list can be repetitive as they are not unique.

Collection data types: List

Username	age	jobs	occupation
Daveg@gmail.com	43	null	engineer
Elaine@yahoo.com	60	Null	producer
dan@gmail.com	46	Null	banker
Alaind@gmail.com	32	['Netflix']	actor
Peterd@gmail.com	32	Null	producer
Moirad@yahoo.com	35	Null	dj

```
UPDATE users SET jobs = jobs - ['Reiss']
WHERE username = 'Alaind@gmail.com';

SELECT * FROM users
WHERE username = 'Alaind@gmail.com';
```

- A select in CQLSH on our table shows that we currently have two jobs recorded for our user. We can also remove an entry from the list. A select on our table now shows that we only have one job listed for our user. We have seen that collection data types can be used instead of a one-to-many join relationship.

User-defined data types (UDTs)

- Can attach multiple data fields, each named and typed, to a single column
- The fields used to create a UDT may be any valid data type, including collections and other existing UDTs
- Once created, the user can alter, verify, and drop a field or the whole data type
- Once created, UDTs may be used to define a column in a table

- However, for one-to-one relationships we can use Cassandra user defined types or UDTs. UDTs can attach multiple data types to a single column. Think of an address being a combination of an apartment number, building, street, and so on. CQL enables the user to create their own data type. The fields used in a created UDT can be any valid data type, including collections and other existing UDTs. After creating a data type and fields in it, the user can alter, verify, and even drop a field or the whole data type. Once created, UDTs can be used to define a column in a table. Let's see how we do this on the next screen.

User-defined data types (UDTs)

```
CREATE TYPE address (  
  Street text,  
  Number int,  
  Flat text);  
  
CREATE TABLE users_w_address(  
  Userid int,  
  Location address,  
  Primary key (userid));  
  
INSERT INTO users_w_address(userid,location) VALUES (1, {street : 'Third', number : 34, flat :  
'34C'}); //insert data  
  
DROP TYPE address; //we can drop a type
```

- In this example, we create a new data type called address. Once created, we can use it to define a column in our new table called location. We can insert data using the new address data type and we can also drop the data type.

Recap

In this video, you learned that:

- Cassandra supports built-in, collection, and user-defined data types
- Both collection and user-defined data types offer a way to group and store data together
- Collection data types can emulate one-to-many relationships
- There are three types of collection data types: Lists, maps, and sets
- UDTs can emulate one-to-one relationships
- UDTs allow users to attach multiple data fields to a column